```python
import math

class Retrieve:

    # Create new Retrieve object storing index and term weighting
    # scheme. (You can extend this method, as required.)
    def __init__(self,index, term_weighting):
        self.index = index                    #inverted index
        self.term_weighting = term_weighting # binary,tf,tf-idf

        #precompute values to avoid repeated calculations
        self.document_count = self.compute_number_of_documents()
        self.idf_values = self.compute_idf()
        self.document_vector_lengths = (self.pre_compute_document_vector_lengths())


#PRE-PROCESSING AND COMPUTATION -------------------------------------------------
    def compute_number_of_documents(self):
        #count total number of distinct documents across the index
        self.doc_ids = set()
        for term in self.index:
            self.doc_ids.update(self.index[term])
        return len(self.doc_ids)

    def compute_idf(self):
        #calcualte idf value for each term in inverted index
        idf_values = {}
        for term, doc_freq in self.index.items():
            df = len(doc_freq)
            idf = math.log((self.document_count/df)) if df > 0 else 0
            idf_values[term] = idf
        return idf_values

    def pre_compute_document_vector_lengths(self):
        #precompute full length of each document vector
        #avoids recomputation during ranking
        document_vector_lengths = {}

        for doc_id in range(1,self.document_count+1):
            vector_length_sum = 0
            for term, doc_freq in self.index.items():
                if doc_id not in doc_freq:
                    continue

                tf = doc_freq[doc_id]

                #apply weighting depending on selected scheme
                if self.term_weighting == 'tf':
                    # weight = tf for raw tf
                    if tf > 0:
                        weight = 1 + math.log(tf)
                    else:
                        weight = 0
                elif self.term_weighting == 'binary':
                    weight = 1
                elif self.term_weighting == 'tfidf':
                    # weight = tf * self.idf_values[term], for raw tf-idf
                    if tf > 0:
                        weight = (1 + math.log(tf)) * self.idf_values[term]
                    else:
                        weight = 0

                #square root the accumlated square weights of terms
                vector_length_sum += weight ** 2
            document_vector_lengths[doc_id] = math.sqrt(vector_length_sum)

        return document_vector_lengths

#--------------------------------------------------------------------------------

#FOR_QUERY MAIN METHOD - CALLED FOR EACH QUERY -----------------------------
    def for_query(self, query):
        #the 'main' method of the class.

        #convert query into a dictionary of term counts
        query_term_counts = self.compute_query_term_counts(query)

        scores = {}

        #calcualte the cosine score for each document id in the collection.
        for doc_id in range(1,self.document_count+1):
            d_vector = self.compute_document_vector(query_term_counts,doc_id)
            q_vector,d_vector = self.apply_weighting_scheme(
                query_term_counts, d_vector)
            scores[doc_id] = self.cosine_similary(q_vector,d_vector,doc_id)
```

```
        #sort the dictionary of cosine values and return the doc ids of top 10.
        scores_ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)
        scores_top10 = list([doc_id for doc_id, score in scores_ranked[:10]])

        return scores_top10
#-------------------------------------------------------------------------

#QUERY TERM COUNTS FUNCTION-----------------------------------------------
    def compute_query_term_counts(self, query):
        query_term_counts = {}
        #convert query list of terms into a dict with term counts
        for term in query:
            if term in query_term_counts:
                query_term_counts[term]  = query_term_counts[term] + 1
            else:
                query_term_counts[term] = 1
        return query_term_counts
#-------------------------------------------------------------------------


#COMPUTE DOCUMENT VECTOR FUNCTION-----------------------------------------
    def compute_document_vector(self,query_term_counts,doc_id):
        #build a document vector only over query terms
        document_vector = []
        for term in query_term_counts:
            #if a term does not appear in a document, append 0
            document_vector.append(self.index.get(term,{}).get(doc_id,0))
        return document_vector
#-------------------------------------------------------------------------


#CALCULATE WEIGHTED VECTORS FUNCTION--------------------------------------
    def apply_weighting_scheme(self,q_dict,d_vector):
        #extract raw query vector and term list
        q_vector = list(q_dict.values())
        q_vector_terms = list(q_dict.keys())


        if self.term_weighting == 'binary':
            #convert to a vector  of 1s and 0s.
            q_vector = [1 if x > 0 else 0 for x in q_vector]
            d_vector = [1 if x > 0 else 0 for x in d_vector]

        elif self.term_weighting == 'tf':
            #apply log normalised tf to each value
            # d_vector = [x if x > 0 else 0 for x in d_vector]
            d_vector = [self.log_tf(x) for x in d_vector]
            q_vector = [self.log_tf(x) for x in q_vector]

        elif self.term_weighting == 'tfidf':
            #apply log normalizsed tf and multiply by idf value
            for term_index in range(0,len(q_vector)):
                idf = self.idf_values.get(q_vector_terms[term_index], 0)
                # d_vector[term_index] *= idf
                # q_vector[term_index] *= idf

                d_tf = self.log_tf(d_vector[term_index])
                d_vector[term_index] = d_tf * idf

                q_tf = self.log_tf(q_vector[term_index])
                q_vector[term_index] = q_tf * idf

        return (q_vector,d_vector)


    def log_tf(self,x):
        #a helper function to make calculating log_tf efficient
        return 1 + math.log(x) if x > 0 else 0


#-------------------------------------------------------------------------

#COMPUTE COSINE SIMILARITY FUNCTION---------------------------------------
    def cosine_similary(self,q_vector,d_vector,doc_id):
        #if the document vector has no related query terms, return score as 0
        if sum(d_vector) != 0:

            #get query vector length and document vector length
            q_length = math.sqrt(sum(x**2 for x in q_vector))
            d_length = self.document_vector_lengths[doc_id]


            #calculate dot product
            dot_product = 0
            for i in range(0,len(q_vector)):
                q_dot_d = q_vector[i] * d_vector[i]
                dot_product += q_dot_d
```

```
            cosine = (dot_product) / (q_length * d_length)

            return cosine

        else:
            return 0

#--------------------------------------------------------------------------
```